

2. Semana 2. Herramientas Computacionales Científicas

Objetivo de la cátedra

Implementar herramientas computacionales fundamentales para análisis numérico, álgebra lineal aplicada, visualización científica y automatización de cálculos, orientadas al desarrollo de simulaciones doctorales reproducibles y verificables.

Temario

- Introducción a Python/MATLAB científico.
- Manejo de arreglos y matrices.
- Visualización científica.
- Scripts reproducibles.
- Introducción a notebooks científicos.
- Librerías científicas.
- Automatización de cálculos.
- Buenas prácticas computacionales.

Propósito del tema. En la Semana 1 se estableció la cadena fenómeno → modelo → simulación → prototipo → validación. Esta semana se construye la infraestructura técnica que permite recorrer esa cadena de manera disciplinada: lenguajes científicos, estructuras de datos, álgebra lineal numérica, visualización rigurosa, automatización y control de versiones. El objetivo no es coleccionar comandos, sino consolidar un entorno donde cualquier resultado pueda ser ejecutado de nuevo —por el autor, por un revisor o por el propio autor seis meses después— y producir exactamente el mismo número, hasta el último bit relevante.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) construir un proyecto computacional reproducible con control de versiones, entorno congelado y semilla aleatoria explícita; (ii) operar arreglos multidimensionales y matrices dispersas con criterios de costo computacional y estabilidad numérica; (iii) producir visualizaciones científicas con tipografía consistente, escala adecuada y exportación vectorial; (iv) automatizar flujos de cálculo mediante `argparse`, archivos de configuración y orquestadores; y (v) seleccionar entre Python y MATLAB con criterios técnicos en lugar de inercia personal, asumiendo que la herramienta es un medio, no una identidad académica.

2.1. Motivación: del lápiz al ecosistema reproducible

Una computadora científica moderna no es una calculadora más rápida; es un *ecosistema* con capas de abstracción que deben dominarse para que un resultado sea defendible. Las capas mínimas que se atravesarán en este curso son:

1. **Lenguaje:** Python 3.x o MATLAB R2024b, con sintaxis y semántica numérica conocidas.
2. **Tipos numéricos:** float32, float64, complex128; aritmética IEEE 754.
3. **Estructuras de datos:** arreglos densos ndarray, arreglos dispersos (CSR/CSC), DataFrame, grafos.
4. **Bibliotecas numéricas:** BLAS, LAPACK, FFTW (subyacentes en NumPy/SciPy/MATLAB).
5. **Bibliotecas científicas de alto nivel:** SciPy, scikit-learn, SymPy, pandas.
6. **Capa de visualización:** Matplotlib, Plotly, ParaView, PyVista.
7. **Capa de orquestación:** scripts, notebooks, argparse, Make/Snakemake, contenedores.
8. **Capa de gobernanza:** Git, entornos virtuales, archivos de *lock*, DOIs de software.

Ignorar cualquiera de estas capas produce resultados frágiles: un *notebook* sin semilla, una figura sin unidades, un `requirements.txt` sin versiones fijas o una matriz mal condicionada son fallos del mismo tipo. En investigación doctoral, la diferencia entre un hallazgo y una anécdota suele residir en estas capas.

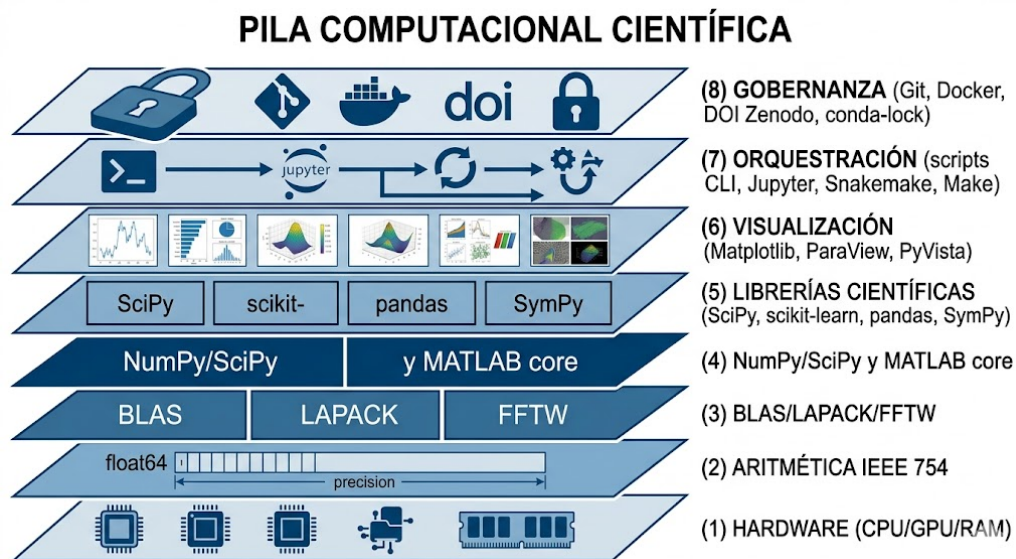


Figura 13: Pila computacional científica utilizada en el curso. Cada capa abstrae la inferior y expone una interfaz a la superior. La capa más alta (gobernanza) garantiza que las inferiores sean repetibles.

2.2. Python y MATLAB: criterios de elección

Ambos lenguajes son herramientas legítimas de investigación. La elección no debe basarse en preferencia estética ni en lo que “se usa en el laboratorio”, sino en restricciones técnicas concretas: licenciamiento, comunidad, integración con hardware, disponibilidad de bibliotecas y portabilidad del proyecto a un eventual repositorio público.

Tabla 8: Comparación técnica entre Python científico y MATLAB para investigación doctoral.

Criterio	Python (NumPy/SciPy)	MATLAB R2024b
Licenciamiento	Open source (BSD/PSF). Sin costo.	Comercial, licencia institucional o de pago.
Indexado	Base 0; $a[i:j]$ excluye j .	Base 1; $a(i:j)$ incluye j .
Asignación	Por referencia ($b=a$ comparte memoria).	Copy-on-write (más predecible para principiantes).
Álgebra lineal	<code>numpy.linalg</code> , <code>scipy.linalg</code> (LAPACK).	Operadores nativos $A \setminus b$, <code>eig</code> , <code>svd</code> .
Ecosistema ML/IA	PyTorch, TensorFlow, JAX, scikit-learn.	Deep Learning Toolbox.
Visualización	Matplotlib, Plotly, PyVista.	Toolboxes nativos, App Designer.
Toolboxes específicos	PyPI: > 500 000 paquetes.	Toolboxes oficiales certificados (Simulink, Stereo Camera Calibrator, etc.).
Reproducibilidad pública	Excelente (pip/conda + Git + Colab).	Limitada por licencia; mitigable con MATLAB Online o Octave.
Comunidad doctoral	Predominante en CS, óptica, ML, física computacional.	Predominante en control, comunicaciones, biomédica.

Recomendación operativa del curso. Se trabajará principalmente en Python 3.11+ con NumPy, SciPy, Matplotlib y pandas, ejecutado en Google Colab para reproducibilidad pública. MATLAB se usará cuando una tarea específica lo exija (por ejemplo, toolboxes de calibración estereoscópica). El estudiante debe ser capaz de traducir un algoritmo entre ambos sin perder fidelidad numérica, lo cual es una habilidad doctoral mínima.

2.2.1. Aritmética IEEE 754: lo que el lenguaje no oculta

Tanto Python como MATLAB usan, por defecto, aritmética de punto flotante de doble precisión (float64, 64 bits, 1 de signo, 11 de exponente, 52 de mantisa). Esto implica:

- Épsilon de máquina $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.
- Rango representable aproximado: $\pm 10^{-308}$ a $\pm 10^{308}$.
- La suma no es asociativa: $(a + b) + c \neq a + (b + c)$ en general.
- Comparar floats con `==` es una forma elegante de introducir bugs. Se debe usar tolerancia: `np.isclose(a,b,atol=1e-9,rtol=1e-7)`.

La presencia de números NaN e Inf es una herramienta de diagnóstico: si aparecen sin haber sido planeados, hay división por cero, log de número no positivo o desbordamiento; nunca son ruido inocente.

2.3. Manejo de arreglos y matrices

El bloque numérico fundamental es el arreglo multidimensional. En NumPy, ndarray es un buffer contiguo de memoria con metadatos (*shape*, *dtype*, *strides*). En MATLAB, todo es matriz; un escalar es una matriz 1×1 .

2.3.1. Vectorización frente a bucles

La diferencia de rendimiento entre código vectorizado y código con bucles explícitos puede ser de uno a tres órdenes de magnitud. Sea $y_i = \sin(x_i) e^{-x_i^2}$, $i = 1, \dots, N$. En Python:

```

1      import numpy as np
2      import time
3
4      N = 10_000_000
5      x = np.linspace(-3.0, 3.0, N)
6
7      # Version 1: bucle for explícito (lento)
8      t0 = time.perf_counter()
9      y_loop = np.empty(N)
10     for i in range(N):
11         y_loop[i] = np.sin(x[i]) * np.exp(-x[i]**2)
12     t_loop = time.perf_counter() - t0
13
14     # Version 2: vectorizada (rapida)
15     t0 = time.perf_counter()
16     y_vec = np.sin(x) * np.exp(-x**2)
17     t_vec = time.perf_counter() - t0
18
19     print(f"Bucle:          {t_loop:8.4f} s")
20     print(f"Vectorizado: {t_vec:8.4f} s")
21     print(f"Speedup:        {t_loop/t_vec:6.1f}x")
22     print(f"Coinciden:    {np.allclose(y_loop, y_vec)}")
23
    
```

Listing 6: Vectorización vs bucle explícito en NumPy.

En una máquina contemporánea típica el *speedup* ronda 50–200×. La razón es que las operaciones vectorizadas se ejecutan en bucles compilados de C apoyados en BLAS/-

SIMD, mientras que el bucle Python paga el costo del intérprete por cada iteración. La regla práctica: *si se está escribiendo un for sobre índices de un ndarray, probablemente hay una mejor manera.*

2.3.2. Broadcasting

Broadcasting es la regla por la cual NumPy combina arreglos de formas distintas sin replicar memoria. Sea $A \in \mathbb{R}^{M \times N}$ y $b \in \mathbb{R}^N$. La operación $A - b$ resta b a cada fila de A sin construir explícitamente una matriz $M \times N$ con b repetido M veces. Las reglas formales son:

1. Las formas se alinean por la derecha.
2. En cada eje, las dimensiones deben ser iguales o una de ellas debe ser 1.
3. Las dimensiones de tamaño 1 se “estiran” virtualmente.

$$A_{i,j} - b_j \quad \text{con} \quad A \in \mathbb{R}^{M \times N}, b \in \mathbb{R}^N \Rightarrow A - b \in \mathbb{R}^{M \times N}. \quad (87)$$

Este mecanismo es la base de gran parte del código vectorizado eficiente en simulación de campos, procesamiento de imágenes y operaciones tensoriales en redes neuronales.

2.3.3. Estructura de memoria y costo computacional

Un ndarray tiene orden de memoria *row-major* (C, por defecto) o *column-major* (Fortran, igual que MATLAB). Recorrer un arreglo en orden contrario al *layout* de memoria multiplica el costo por factores de 2–10 por errores de caché. Para operaciones críticas:

- Verificar `a.flags['C_CONTIGUOUS']` o `a.flags['F_CONTIGUOUS']`.
- Forzar contigüidad con `np.ascontiguousarray(a)` si se invocará BLAS.
- Evitar *views* no contiguos en bucles internos.

La complejidad asintótica de operaciones comunes en una matriz $N \times N$ densa se resume en la [Tabla 9](#).

Tabla 9: Complejidad asintótica y biblioteca subyacente de operaciones de álgebra lineal densa.

Operación	Complejidad	Rutina LAPACK/BLAS
Producto matriz–vector Ax	$\mathcal{O}(N^2)$	DGEMV (BLAS-2)
Producto matriz–matriz AB	$\mathcal{O}(N^3)$	DGEMM (BLAS-3)
Resolución $Ax = b$ (LU)	$\mathcal{O}(N^3)$	DGESV
Cholesky (SPD)	$\mathcal{O}(N^3/3)$	DPOTRF
QR	$\mathcal{O}(2N^3)$	DGEQRF
SVD completa	$\mathcal{O}(N^3)$	DGESDD
Eigenvalores (no simétrico)	$\mathcal{O}(\sim 10N^3)$	DGEEV
FFT 1D longitud N	$\mathcal{O}(N \log N)$	FFTW / pocketfft

2.3.4. Matrices dispersas

En problemas de elementos finitos, mallas grandes, redes o sistemas estructurados, la matriz K es predominantemente cero. Almacenarla densa para $N = 10^5$ requiere $N^2 \cdot 8 \text{ bytes} \approx 80 \text{ GB}$, lo cual es absurdo. Se usan formatos dispersos: LIL para ensamblado, CSR/CSC para resolución, COO para entrada/salida. La resolución se hace con `scipy.sparse.linalg.spsolve` (solvers directos, hasta $\sim 10^6$ incógnitas en RAM razonable) o `cg`, `gmres`, `bicgstab` (iterativos, para problemas mayores).

El Ejemplo 8 de la Semana 1 (conexión viga–columna con FEM CST) ya usaba este patrón: ensamblado en `lil_matrix` y resolución en `csr_matrix` con `spsolve`. Esa elección no es estética; es la diferencia entre un código que corre y un código que agota la memoria.

2.4. Visualización científica

Una figura no es decoración: es un argumento técnico. En contexto doctoral, una figura defectuosa puede invalidar conclusiones correctas. Los criterios mínimos que se exigirán en este curso son:

1. **Unidades en todos los ejes**, sin excepción.
2. **Etiquetas legibles** a tamaño impreso (mínimo 9 pt para revistas, 12 pt para presentaciones).
3. **Mapas de color perceptualmente uniformes**: `viridis`, `plasma`, `inferno`, `cividis`. Nunca `jet` ni `rainbow` para datos cuantitativos (introducen falsos bordes y son inaccesibles para daltónicos).
4. **Aspect ratio físico** cuando se grafican dominios espaciales: `ax.set_aspect('equal')`.
5. **Exportación vectorial**: `.pdf`, `.svg` o `.eps` para figuras de líneas; `.png` a ≥ 300 dpi para mapas densos.

6. **Tipografía consistente** con el documento final (Times, Computer Modern o sans-serif uniformes).

2.4.1. Plantilla mínima “publication-ready”

```

1      import matplotlib.pyplot as plt
2      import matplotlib as mpl
3
4      # ---- Configuración global para figuras de
publicacion ----
5      mpl.rcParams.update({
6          "font.family":      "serif",
7          "font.size":        11,
8          "axes.titlesize":   12,
9          "axes.labelsize":   11,
10         "xtick.labelsize":   10,
11         "ytick.labelsize":   10,
12         "legend.fontsize":   10,
13         "axes.linewidth":    0.8,
14         "lines.linewidth":   1.4,
15         "figure.dpi":        150,
16         "savefig.dpi":       300,
17         "savefig.bbox":      "tight",
18         "savefig.format":    "pdf",
19         "text.usetex":       False,    # True si hay
LaTeX instalado
20         "mathtext.fontset":  "cm",
21     })
22

```

Listing 7: Plantilla de configuración de Matplotlib para figuras científicas.

Esta plantilla se incluye una sola vez al inicio del proyecto. Toda figura generada después hereda el estilo, garantizando consistencia visual sin esfuerzo manual.

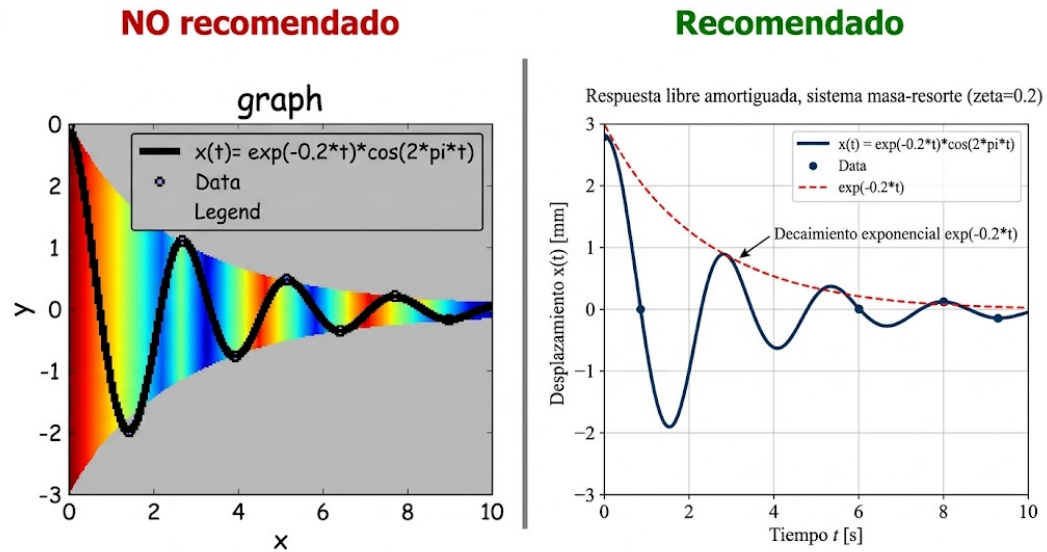


Figura 14: Contraste entre una figura defectuosa (izquierda) y una figura “publication-ready” (derecha) representando los mismos datos. La diferencia no es cosmética: la figura defectuosa oculta la información cuantitativa que el revisor necesita evaluar.

2.4.2. Mapas de color y percepción

El uso de jet (arcoíris) sigue siendo común y sigue siendo incorrecto para datos cuantitativos. Las razones son:

- No es perceptualmente uniforme: pasos iguales en el dato producen saltos visuales de magnitud distinta.
- Introduce bandas falsas en transiciones amarillo–cian que el ojo interpreta como contornos físicos inexistentes.
- No es robusto a impresión en escala de grises.
- Inaccesible para parte de la población con visión cromática reducida.

La familia viridis (y derivados plasma, inferno, magma, cividis) fue diseñada para corregir estos cuatro defectos simultáneamente. En este curso, jet está prohibido en entregables.

2.5. Scripts reproducibles y arquitectura de proyectos

Un *script* reproducible es aquel que, ejecutado por un tercero con los mismos datos de entrada y los mismos parámetros, produce los mismos resultados. La estructura mínima recomendada para un proyecto doctoral es:

```

1      mi_proyecto/
2      |-- README.md          # objetivo,
      instalacion, ejecucion
    
```



```

3         |-- LICENSE                                # licencia explicita (
MIT, BSD, GPL, CC-BY)
4         |-- environment.yml                        # conda env o
requirements.txt
5         |-- .gitignore                            # excluye datos crudos
pesados, __pycache__
6         |-- src/                                  # codigo fuente Python
/MATLAB
7         | |-- __init__.py
8         | |-- modelo.py
9         | |-- solver.py
10        | |-- utils.py
11        |-- scripts/                              # entradas de linea de
comandos
12        | |-- run_experiment.py
13        | |-- make_figures.py
14        |-- config/                                # parametros por
experimento (yaml/json)
15        | |-- exp_001.yaml
16        | |-- exp_002.yaml
17        |-- data/
18        | |-- raw/                                # datos crudos (read-
only, idealmente con DOI)
19        | |-- processed/                          # derivados,
regenerables
20        |-- results/                              # salidas: figuras,
tablas, logs
21        |-- tests/                                # pruebas unitarias (
pytest)
22        |-- notebooks/                            # exploracion, NUNCA
es la fuente de verdad
23

```

Listing 8: Estructura mínima de directorio para un proyecto doctoral reproducible.

Esta jerarquía es la *convención de facto* en proyectos científicos modernos (similar a Cookiecutter Data Science) y se enseña porque elimina ambigüedades clásicas: dónde están los datos, dónde el código, dónde los resultados, qué se puede borrar sin pérdida.

2.5.1. Script ejecutable con argparse y registro de parámetros

```

1         """
2         run_diffusion.py

```

```
3      Simulacion 1D de difusion termica con parametros
4      configurables
5      desde linea de comandos y registro automatico de
6      configuracion.
7
8      Uso:
9      python run_diffusion.py --alpha 1e-4 --N 151 --r
10     0.45 \
11     --tsteps 800 --seed 42 \
12     --outdir results/exp_001
13     """
14
15     import argparse
16     import json
17     import os
18     import sys
19     import time
20     from pathlib import Path
21
22     import numpy as np
23     import matplotlib.pyplot as plt
24
25
26     def parse_args():
27         p = argparse.ArgumentParser(description="Difusion 1D
28         explícita.")
29         p.add_argument("--L",          type=float, default=1.0,
30         help="Longitud [m]")
31         p.add_argument("--alpha",      type=float, default=1e-4,
32         help="Difusividad [m^2/s]")
33         p.add_argument("--N",          type=int,   default=151,
34         help="Nodos espaciales")
35         p.add_argument("--r",          type=float, default=0.45,
36         help="Numero de Fourier <= 0.5")
37         p.add_argument("--tsteps",     type=int,   default=800,
38         help="Pasos temporales")
39         p.add_argument("--T0",         type=float, default=25.0,
40         help="Temperatura ambiente [C]")
41         p.add_argument("--A",          type=float, default=80.0,
42         help="Amplitud gaussiana [C]")
43         p.add_argument("--seed",       type=int,   default=42,
44         help="Semilla aleatoria")
```

```

32         p.add_argument("--outdir", type=str, default="
results/run_default")
33         return p.parse_args()
34
35
36     def main():
37         args = parse_args()
38         outdir = Path(args.outdir)
39         outdir.mkdir(parents=True, exist_ok=True)
40
41         # ---- Snapshot de configuracion ----
42         cfg = vars(args).copy()
43         cfg["timestamp"] = time.strftime("%Y-%m-%dT%H:%M:%
S")
44
45         cfg["numpy"] = np.__version__
46         cfg["python"] = sys.version.split()[0]
47         with open(outdir / "config.json", "w") as f:
48             json.dump(cfg, f, indent=2)
49
50         # ---- Verificacion de estabilidad ----
51         if args.r > 0.5:
52             sys.exit(f"ERROR: r = {args.r} > 0.5; el esquema
53             explícito sera inestable.")
54
55         # ---- Simulacion ----
56         np.random.seed(args.seed)
57         x = np.linspace(0.0, args.L, args.N)
58         dx = x[1] - x[0]
59         dt = args.r * dx**2 / args.alpha
60
61         T = args.T0 + args.A * np.exp(-((x - 0.5*args.L)**2)
62         / (2*0.06**2))
63         T[0] = T[-1] = args.T0
64
65         snapshots, times = [T.copy()], [0.0]
66         for n in range(1, args.tsteps + 1):
67             T[1:-1] = T[1:-1] + args.r * (T[2:] - 2*T[1:-1] + T
68            [:-2])
69             T[0] = T[-1] = args.T0
70             if n in (50, 150, 300, args.tsteps):
71                 snapshots.append(T.copy())
72                 times.append(n * dt)

```

```

69
70     # ---- Salida tabular ----
71     np.savez(outdir / "fields.npz", x=x, T=np.stack(
snapshots), t=np.array(times))
72
73     # ---- Figura ----
74     fig, ax = plt.subplots(figsize=(7, 4))
75     for Tk, tk in zip(snapshots, times):
76         ax.plot(x, Tk, label=f"t = {tk:6.2f} s")
77         ax.set_xlabel("x [m]")
78         ax.set_ylabel("T [degC]")
79         ax.set_title("Conduccion transitoria 1D")
80         ax.grid(True, alpha=0.3)
81         ax.legend()
82     fig.tight_layout()
83     fig.savefig(outdir / "perfiles.pdf")
84
85     print(f"[OK] resultados en {outdir.resolve()}")
86
87
88     if __name__ == "__main__":
89         main()
90

```

Listing 9: Plantilla de script reproducible con argparse y registro de parámetros.

Tres elementos distinguen este script de un *notebook* desechable:

1. Toda variable de control es un argumento explícito; nada está *hardcoded*.
2. Se guarda un `config.json` con la versión de Python, NumPy, semilla y *timestamp*; esto permite reconstruir el experimento meses después.
3. Verificación de estabilidad ($r \leq 0.5$) integrada como condición de salida con código de error, no como advertencia ignorable.

2.6. Notebooks científicos: cuándo sí y cuándo no

Jupyter (y su variante en la nube Google Colab) es excelente para exploración, didáctica, comunicación visual y revisión interactiva. Es *malo* como fuente de verdad de un proyecto serio. La regla operativa es:

- **Usar notebooks para:** exploración inicial, prototipos de figuras, tutoriales reproducibles compartidos con coautores, material docente.
- **No usar notebooks para:** ejecutar pipelines de producción, almacenar la versión canónica de funciones reutilizables, controlar versiones de cálculos críticos.

Los problemas conocidos de los notebooks son: orden de ejecución arbitrario (un cuaderno se ejecuta de arriba abajo sólo si el usuario lo decide), *diffs* de Git ilegibles por el JSON con salidas embebidas, dependencias ocultas en el *kernel* de la sesión y dificultad para parametrizar. Herramientas que mitigan estos defectos: *nbstripout* (limpia salidas antes de *commit*), *papermill* (parametriza notebooks como si fueran scripts), *jupyter-text* (sincroniza con `.py` versionables).

2.6.1. Google Colab: rol específico en el curso

Colab se usará como entorno de *demonstración pública y reproducible* de los ejemplos del curso porque: (i) no requiere instalación local, (ii) la URL es portable, (iii) ya tiene NumPy, SciPy, Matplotlib y PyTorch preinstalados, (iv) ofrece GPU gratuita para entrenamientos cortos. Los entregables formales, sin embargo, serán scripts versionados en Git, no notebooks de Colab.

2.7. Librerías científicas fundamentales

El ecosistema Python científico se construye sobre cinco librerías base, sobre las cuales se montan especializaciones por dominio.

Tabla 10: Librerías científicas usadas durante el curso, con su rol y dependencia subyacente.

Librería	Rol principal	Subyacente / notas
numpy	Arreglos multidimensionales, álgebra lineal básica.	BLAS/LAPACK.
scipy	Optimización, integración, ODE/PDE, dispersas, estadística.	FORTTRAN legado (quadpack, odepack).
matplotlib	Visualización 2D y 3D estática.	Backend Agg/Cairo.
pandas	Tablas indexadas, series temporales, I/O.	NumPy + C++.
sympy	Álgebra simbólica, derivadas analíticas, simplificación.	Python puro.
scikit-learn	Aprendizaje automático clásico.	NumPy/SciPy.
scikit-image	Procesamiento de imágenes científicas.	NumPy.
statsmodels	Inferencia estadística clásica.	NumPy/SciPy.
torch / tensorflow	Redes neuronales, diferenciación automática.	CUDA/cuDNN.
pyvista / vtk	Visualización 3D y mallas.	VTK.
numba / cython	Aceleración JIT/compilada.	LLVM, C.

2.7.1. SymPy: cuándo lo simbólico ahorra tiempo

SymPy permite obtener derivadas analíticas, simplificar expresiones y generar código numérico desde una formulación matemática. Su valor doctoral aparece cuando se necesita el Jacobiano $J = \partial f / \partial x$ para propagación de incertidumbre (Tabla 9) o para esquemas de Newton sin riesgo de error humano en la derivada.

```

1      import sympy as sp
2
3      I, R, h, As, t, m, cp, Tinf = sp.symbols("I R h As t
4      m cp Tinf", positive=True)
5
6      # Modelo concentrado de calentamiento (Semana 1,
7      Ejemplo 3)
8      tau = m*cp / (h*As)
9      Tss = Tinf + I**2*R / (h*As)
10     T = Tinf + I**2*R/(h*As) * (1 - sp.exp(-t/tau))
11
12     # Jacobiano respecto a (I, R, h)
13     J = sp.Matrix([T]).jacobian([I, R, h])
14     J_simpl = sp.simplify(J)
15     sp.pprint(J_simpl)
16
17     # Exportar a funcion numerica vectorizada
18     T_num = sp.lambdify((I, R, h, As, t, m, cp, Tinf), T
19     , modules="numpy")
20     J_num = sp.lambdify((I, R, h, As, t, m, cp, Tinf),
21     J_simpl, modules="numpy")
22
23     # Evaluacion numerica
24     print(T_num(400.0, 1e-4, 25.0, 0.5, 60.0, 2.0,
25     900.0, 25.0))

```

Listing 10: Derivación simbólica del Jacobiano y exportación a función numérica.

Este patrón —derivar simbólicamente, *lambdificar* para evaluación numérica— es la forma profesional de evitar errores algebraicos. Pertenece al mismo paradigma de la diferenciación automática (autograd, `jax`, `torch.autograd`) que se verá en la Semana 14.

2.8. Automatización de cálculos

Una vez que un experimento computacional se ha estabilizado, debe poder ejecutarse sin intervención manual. Las técnicas escalan desde `Makefile` (suficiente para proyectos

pequeños) hasta orquestadores como Snakemake, Nextflow o Apache Airflow (necesarios cuando hay decenas de pasos interdependientes).

2.8.1. Makefile mínimo para un proyecto científico

```

1      # Makefile - Proyecto Difusion 1D
2      PYTHON := python
3      SRC    := scripts/run_diffusion.py
4      RESULTS := results
5
6      EXPS := exp_001 exp_002 exp_003
7
8      .PHONY: all clean figures
9
10     all: $(EXPS:%=$(RESULTS)/%/perfiles.pdf) figures
11
12     $(RESULTS)/exp_001/perfiles.pdf:
13     $(PYTHON) $(SRC) --alpha 1e-4 --r 0.45 --outdir $(
RESULTS)/exp_001
14
15     $(RESULTS)/exp_002/perfiles.pdf:
16     $(PYTHON) $(SRC) --alpha 5e-5 --r 0.40 --outdir $(
RESULTS)/exp_002
17
18     $(RESULTS)/exp_003/perfiles.pdf:
19     $(PYTHON) $(SRC) --alpha 2e-4 --r 0.45 --outdir $(
RESULTS)/exp_003
20
21     figures: $(EXPS:%=$(RESULTS)/%/perfiles.pdf)
22     $(PYTHON) scripts/make_figures.py --indirs $(EXPS:%=
$(RESULTS)/%)
23
24     clean:
25     rm -rf $(RESULTS)/exp_*
26

```

Listing 11: Makefile para automatizar tres experimentos con dependencias declaradas.

Con un solo comando `make all` se ejecutan los tres experimentos y se generan las figuras consolidadas. `make` sólo re-ejecuta lo que cambió: si se modifica `exp_002.yaml`, sólo se regenera `exp_002`. Esto evita el clásico “ejecuté todo de nuevo por si acaso” que consume horas y oculta errores.

2.9. Buenas prácticas computacionales

Estas prácticas son las que diferencian, en evaluación doctoral, un trabajo robusto de uno frágil:

1. **Control de versiones (Git):** todo proyecto bajo Git desde el primer día. *Commits* pequeños, mensajes descriptivos. Cualquier resultado reportado debe ser trazable a un commit específico.
2. **Entorno aislado:** conda o venv con `environment.yml/requirements.txt` que fije versiones (`numpy==1.26.4`, no `numpy>=1.26`).
3. **Semilla aleatoria explícita:** `np.random.seed(SEED)` al inicio de cualquier proceso estocástico. Sin semilla no hay reproducibilidad.
4. **Aserciones y verificación:** `assert` en condiciones invariantes (estabilidad, conservación de masa/energía, dimensiones de matrices). Estas aserciones son auto-documentación.
5. **Pruebas unitarias:** `pytest` sobre funciones críticas. Una función numérica sin al menos una prueba contra solución analítica o caso límite es código no validado.
6. **Estilo de código:** PEP 8 en Python, formateador automático (`black`, `ruff`). MATLAB tiene `checkcode`. No es estética: el código consistente reduce la carga cognitiva del revisor.
7. **Documentación inline:** *docstrings* en cada función, con descripción, parámetros, retornos, unidades y referencia bibliográfica si aplica.
8. **Logging frente a print:** usar logging con niveles `DEBUG/INFO/WARNING/ERROR`. Un `print` se ignora; un `WARNING` aparece en `stderr` y es trazable.

CICLO DE VIDA DE UN PROYECTO CIENTIFICO REPRODUCIBLE

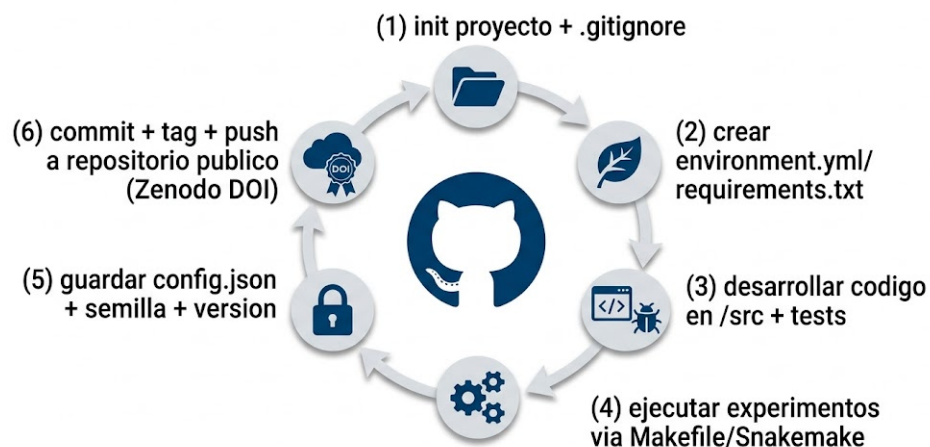


Figura 15: Ciclo de vida de un proyecto científico reproducible. Cada etapa produce un artefacto versionable y trazable, y ninguna depende de memoria humana ni de configuraciones implícitas.

2.9.1. Mini-guía de Git para investigación

```

1      # Inicializacion
2      git init
3      git add README.md environment.yml .gitignore
4      git commit -m "init: estructura inicial del proyecto"
5
6      # Flujo diario
7      git status                    # que ha cambiado
8      git diff src/modelo.py        # ver cambios concretos
9      git add src/modelo.py
10     git commit -m "fix: corregir signo en termino de
11     conveccion"
12
13     # Marcar version reproducible
14     git tag -a v1.0-paper -m "Version usada en el
15     manuscrito enviado a OLE"
16     git push --tags origin main
17
18     # Reproducir un resultado pasado
19     git checkout v1.0-paper        # vuelve al estado
20     exacto del commit

```

Listing 12: Comandos esenciales de Git para investigación reproducible.

Un proyecto sin Git no es defectuoso: es no-investigación. La pregunta “¿qué versión del código produjo esta figura?” debe tener una respuesta exacta (un hash de commit de 40 caracteres), no aproximada.

2.10. Ejemplo 1: Álgebra lineal aplicada — SVD y reducción de dimensionalidad en datos experimentales

2.10.1. Planteamiento

Considere m mediciones experimentales, cada una representada por un vector de n variables: $X \in \mathbb{R}^{m \times n}$. Cuando n es grande (espectros, imágenes, series temporales sincronizadas), conviene reducir la dimensionalidad preservando la varianza. La herramienta es la descomposición en valores singulares (SVD):

$$X = U \Sigma V^T, \quad U \in \mathbb{R}^{m \times m}, \Sigma \in \mathbb{R}^{m \times n}, V \in \mathbb{R}^{n \times n}. \quad (88)$$

Los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ cuantifican cuánta varianza captura cada componente. El análisis de componentes principales (PCA) es la SVD aplicada sobre X centrado.

2.10.2. Conexión doctoral

Este procedimiento aparece directamente en metrología óptica: en ESPI, fringe projection y DIC, los campos de fase o desplazamiento adquiridos en múltiples instantes se organizan en matrices que admiten descomposición SVD para separar señal de ruido (técnica DMD/POD). También subyace la calibración estereoscópica cuando se resuelve $Ax \approx b$ por mínimos cuadrados con pseudoinversa $A^+ = V\Sigma^+U^\top$.

2.10.3. Código en Python

```

1      """
2      Ejemplo 1: SVD aplicada a un conjunto de mediciones
      sintéticas.
3      Se construyen m=200 senales, cada una en n=64 puntos
      , generadas
4      como combinacion lineal de 3 modos fundamentales mas
      ruido gaussiano.
5      El objetivo es recuperar el rango efectivo de los
      datos.
6      """
7      import numpy as np
8      import matplotlib.pyplot as plt
9
10     rng = np.random.default_rng(2026)
11     m, n = 200, 64
12     t = np.linspace(0, 1, n)
13
14     # Modos fundamentales: rango intrinseco r=3
15     phi1 = np.sin(2*np.pi*1*t)
16     phi2 = np.sin(2*np.pi*3*t) * 0.6
17     phi3 = np.cos(2*np.pi*5*t) * 0.3
18     Phi = np.stack([phi1, phi2, phi3], axis=1)      #
      n x 3
19
20     # Coeficientes aleatorios para cada medicion
21     C = rng.normal(size=(m, 3))
22
23     # Datos sin ruido + ruido gaussiano

```

```

24         X_clean = C @ Phi.T                                     #
    m x n
25         sigma_noise = 0.15
26         X = X_clean + sigma_noise * rng.normal(size=(m, n))
27
28         # Centrado (paso obligatorio para PCA)
29         X_mean = X.mean(axis=0)
30         Xc = X - X_mean
31
32         # SVD
33         U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
34
35         # Varianza explicada acumulada
36         var_exp = S**2 / np.sum(S**2)
37         var_cum = np.cumsum(var_exp)
38
39         # Reconstruccion truncada con k=3 componentes
40         k = 3
41         X_rec = U[:, :k] @ np.diag(S[:k]) @ Vt[:k, :] +
X_mean
42
43         err_rec = np.linalg.norm(X - X_rec) / np.linalg.norm
(X)
44         print(f"sigma_1..sigma_5 = {S[:5]}")
45         print(f"Varianza explicada k=3: {var_cum[2]*100:.2f
}%")
46         print(f"Error relativo de reconstruccion: {err_rec
:.4f}")
47
48         # ---- Visualizacion ----
49         fig, axes = plt.subplots(1, 3, figsize=(13, 4))
50
51         ax = axes[0]
52         ax.semilogy(np.arange(1, len(S)+1), S, 'o-', ms=4)
53         ax.axvline(k+0.5, color='r', ls='--', lw=0.8, label=
f'corte k={k}')
54         ax.set_xlabel('indice del modo')
55         ax.set_ylabel(r'$\sigma_i$')
56         ax.set_title('Valores singulares')
57         ax.grid(True, which='both', alpha=0.3); ax.legend()
58
59         ax = axes[1]

```

```

60     ax.plot(np.arange(1, len(S)+1), var_cum*100, 'o-',
ms=4)
61
62     ax.axhline(99, color='r', ls='--', lw=0.8, label='
99%')
63
64     ax.set_xlabel('numero de componentes retenidas')
65     ax.set_ylabel('varianza acumulada [%]')
66     ax.set_title('Curva de varianza')
67     ax.grid(True, alpha=0.3); ax.legend()
68
69     ax = axes[2]
70     idx = 0
71     ax.plot(t, X[idx], 'k.', ms=3, label='medicion
ruidosa')
72     ax.plot(t, X_clean[idx], 'b-', lw=1.2, label='senal
real')
73     ax.plot(t, X_rec[idx], 'r--', lw=1.2, label=f'recon.
SVD k={k}')
74     ax.set_xlabel('t'); ax.set_ylabel('amplitud')
75     ax.set_title(f'Reconstruccion medicion #{idx}')
76     ax.grid(True, alpha=0.3); ax.legend()
77
78     plt.tight_layout()
79     plt.savefig("Figuras/fig_s2_svd_pca.pdf")
80     plt.show()

```

Listing 13: SVD para reducción de dimensionalidad sobre datos sintéticos con ruido.

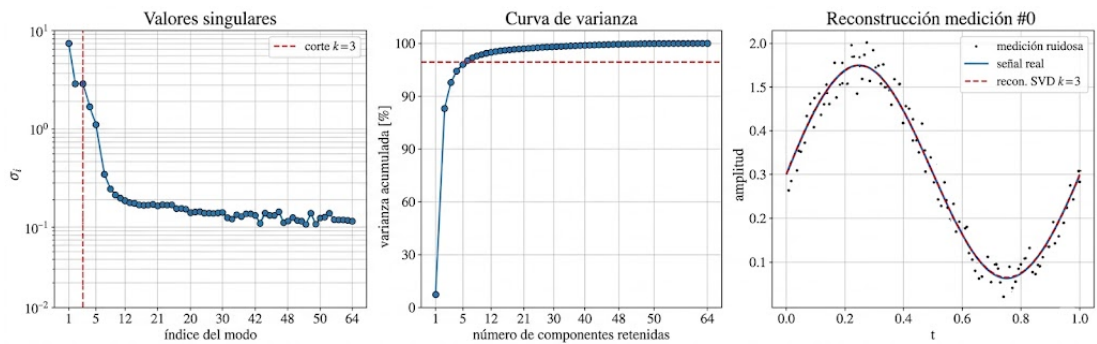


Figura 16: Descomposición en valores singulares de un conjunto sintético de $m = 200$ mediciones en $n = 64$ puntos, con rango intrínseco $r = 3$ y ruido gaussiano $\sigma = 0.15$. Izquierda: espectro de valores singulares, donde los tres primeros modos se separan nítidamente del ruido. Centro: la varianza acumulada alcanza $> 99\%$ con $k = 3$ componentes. Derecha: la reconstrucción truncada filtra el ruido sin sacrificar la señal subyacente.

2.10.4. Lectura metodológica

La SVD ofrece tres cosas simultáneamente: una base ortogonal óptima para representar los datos, un criterio cuantitativo (la curva de valores singulares) para decidir cuántos modos conservar y un mecanismo de *denoising* controlado. La existencia de un “codo” en el espectro de σ_i es una herramienta de diagnóstico: si no hay codo, los datos no tienen estructura de baja dimensionalidad y la reducción es ilegítima.

2.11. Ejemplo 2: Integración de ODE con `scipy.integrate.solve_ivp`

2.11.1. Planteamiento

La integración numérica de ecuaciones diferenciales ordinarias es operación cotidiana en ingeniería. SciPy provee `solve_ivp`, que implementa Runge–Kutta–Fehlberg (RK45), Dormand–Prince, métodos de paso variable y solucionadores rígidos (BDF, Radau, LSODA). Se ilustra con el oscilador de Van der Pol, sistema con dinámica rígida en su régimen relajación:

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0. \quad (89)$$

Para $\mu \gg 1$ el sistema se vuelve rígido y un método explícito de paso fijo (Euler) requiere pasos absurdamente pequeños o explota.

2.11.2. Código en Python

```

1      """
2      Ejemplo 2: integracion del oscilador de Van der Pol.
3      Se comparan tres metodos de solve_ivp: RK45 (
4      explicito no rigido),
5      LSODA (auto-detecta rigidez) y BDF (implicito para
6      rigidez).
7      Se reporta el numero de evaluaciones de f y el
8      tiempo de pared.
9      """
10     import time
11     import numpy as np
12     import matplotlib.pyplot as plt
13     from scipy.integrate import solve_ivp
14
15     def van_der_pol(t, y, mu):
16         x, v = y
17         return [v, mu*(1 - x**2)*v - x]
```

```

17         t_span = (0.0, 100.0)
18         y0 = [2.0, 0.0]
19         mu_values = [1.0, 10.0, 100.0]    # tres regimenes de
20         rigidez
21         metodos = ["RK45", "LSODA", "BDF"]
22
23         print(f"{'mu':>6} {'metodo':>8} {'nfev':>8} {'njev'
24         ':>6} {'tiempo[s]':>10} {'status':>6}")
25         fig, axes = plt.subplots(1, 3, figsize=(13, 4))
26         for ax, mu in zip(axes, mu_values):
27             for met in metodos:
28                 t0 = time.perf_counter()
29                 sol = solve_ivp(van_der_pol, t_span, y0, args=(mu,),
30                 method=met, rtol=1e-6, atol=1e-9,
31                 dense_output=True)
32                 dt = time.perf_counter() - t0
33                 njev = getattr(sol, "njev", 0)
34                 print(f"{'mu':6.0f} {'met':>8} {'sol.nfev':8d} {'njev':6d} {'
35                 dt':10.4f} {'sol.status':>6d}")
36
37                 tt = np.linspace(*t_span, 4000)
38                 yy = sol.sol(tt)
39                 ax.plot(tt, yy[0], lw=1.0, label=met)
40
41                 ax.set_title(f"mu = {mu:.0f}")
42                 ax.set_xlabel("t"); ax.set_ylabel("x(t)")
43                 ax.grid(True, alpha=0.3); ax.legend(fontsize=8)
44
45                 plt.tight_layout()
46                 plt.savefig("Figuras/fig_s2_van_der_pol.pdf")
47                 plt.show()

```

Listing 14: Comparación de métodos en solve_ivp para el oscilador de Van der Pol.

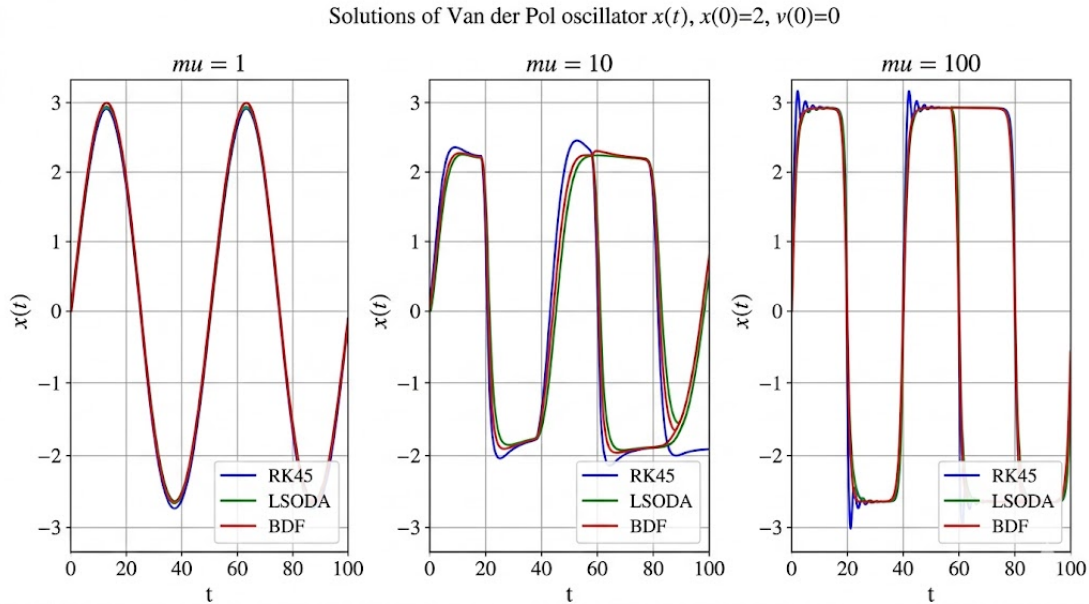


Figura 17: Oscilador de Van der Pol integrado con tres métodos de `solve_ivp`. Para $\mu = 1$ todos los métodos coinciden a costo similar; para $\mu = 100$ el sistema es rígido y los métodos implícitos (BDF, LSODA) requieren órdenes de magnitud menos evaluaciones que RK45.

2.11.3. Lectura metodológica

El número de evaluaciones `nfev` reportado por `solve_ivp` es un diagnóstico directo de costo. Si un método explícito requiere $> 10^5$ evaluaciones donde un implícito requiere 10^3 , el sistema es rígido. Esta detección no debe hacerse por intuición sino por la diferencia observada de costo. La regla doctoral es: si `nfev` crece más rápido que linealmente con el tiempo simulado, hay rigidez y el método está mal elegido.

2.12. Ejemplo 3: pandas para manejo de datos experimentales

2.12.1. Planteamiento

Los datos experimentales rara vez llegan en formato matricial limpio. Suelen ser archivos `.csv` con encabezados inconsistentes, valores faltantes, unidades implícitas y columnas mezcladas. `pandas` es la herramienta estándar para limpiarlos, indexarlos y resumirlos antes del análisis numérico propiamente dicho.

```

1      """
2      Ejemplo 3: pipeline pandas sobre dataset sintetico
      de mediciones
3      de temperatura en N=10 termopares durante 1 hora,
      con perdidas
4      aleatorias (NaN) y outliers.
    
```

```

5         """
6         import numpy as np
7         import pandas as pd
8         import matplotlib.pyplot as plt
9
10        # ---- Generacion de dataset sintetico ----
11        rng = np.random.default_rng(123)
12        n_pts = 3600                                # 1
13        muestra/s durante 1 hora
14        t = pd.date_range("2026-05-21 08:00", periods=n_pts,
15        freq="1s")
16        n_tc = 10
17        T_base = 25 + 30*np.exp(-np.linspace(0, 3, n_pts))
18        # transitorio
19        data = T_base[:, None] + rng.normal(0, 0.4, size=(
20        n_pts, n_tc))
21
22        # Inyectar NaN (perdidas de senal) y outliers (
23        chispas EMI)
24        mask_nan = rng.random((n_pts, n_tc)) < 0.005
25        data[mask_nan] = np.nan
26        mask_out = rng.random((n_pts, n_tc)) < 0.001
27        data[mask_out] += rng.normal(0, 50, mask_out.sum())
28
29        df = pd.DataFrame(data,
30        index=t,
31        columns=[f"TC_{i:02d}" for i in range(n_tc)])
32
33        # ---- Diagnostico ----
34        print("Shape:", df.shape)
35        print("NaN por columna:", df.isna().sum().to_dict())
36        print("Resumen descriptivo:")
37        print(df.describe().T[["mean", "std", "min", "max"
38        ]])
39
40        # ---- Limpieza ----
41        # 1) eliminar outliers via mediana movil + filtro de
42        Hampel
43        median = df.rolling("60s", center=True).median()
44        mad     = (df - median).abs().rolling("60s", center=
45        True).median()
46        score   = (df - median).abs() / (1.4826*mad + 1e-9)

```

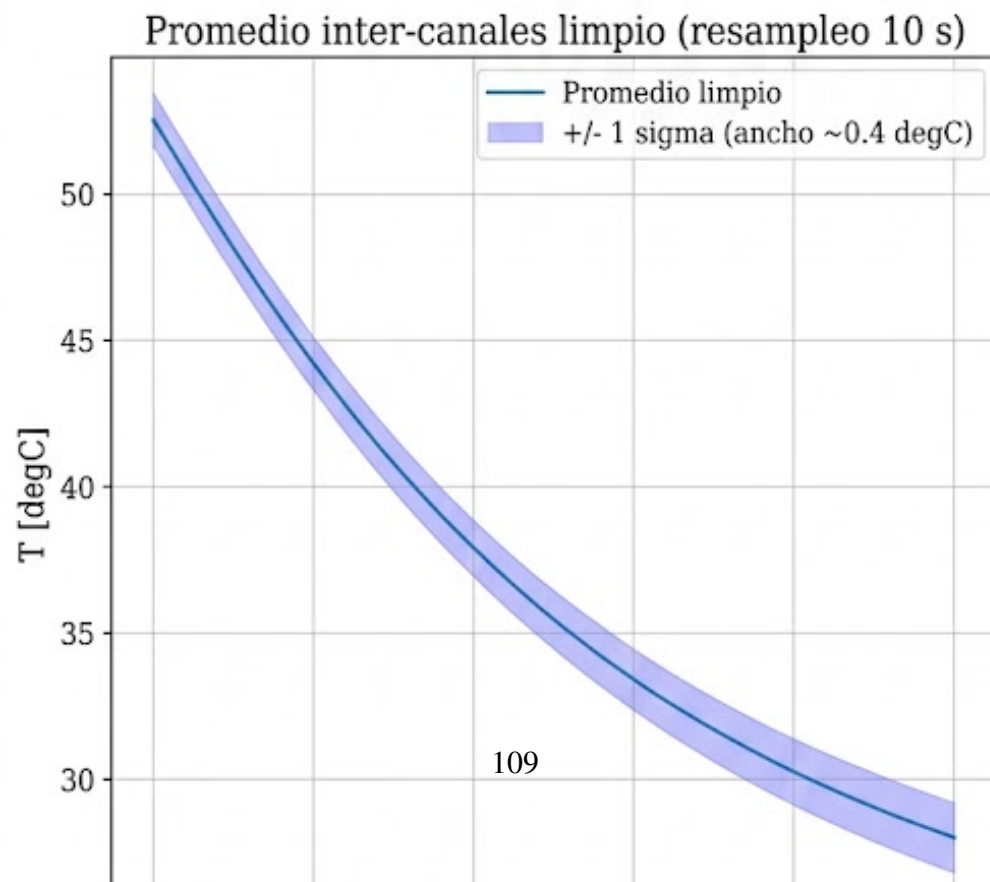
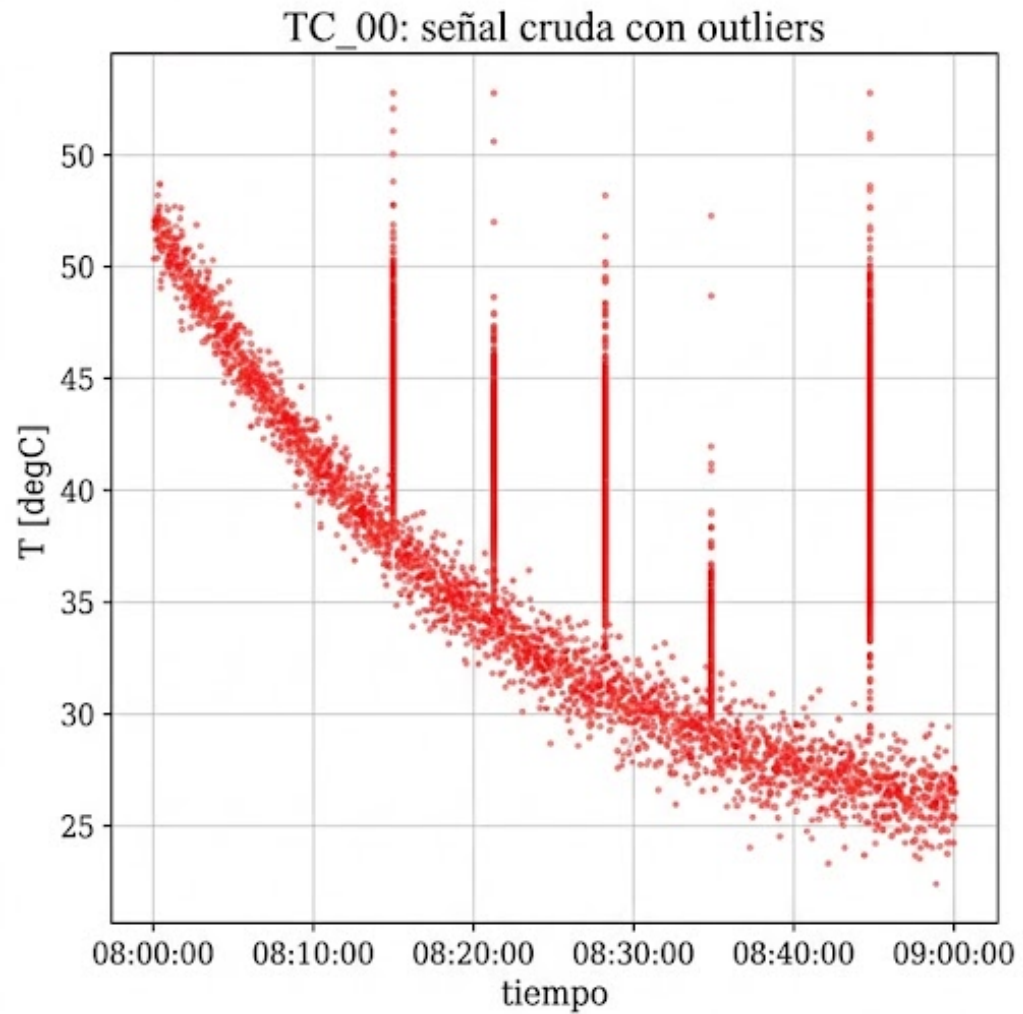


```

39         df_clean = df.where(score < 3.5)
40
41         # 2) imputar NaN por interpolacion temporal
42         df_clean = df_clean.interpolate(method="time", limit
=10)
43
44         # 3) resampleo a 10 s con promedio
45         df_10s = df_clean.resample("10s").mean()
46
47         # ---- Visualizacion ----
48         fig, axes = plt.subplots(2, 1, figsize=(9, 6),
sharex=True)
49         axes[0].plot(df.index, df["TC_00"], 'r.', ms=2,
alpha=0.5, label="crudo")
50         axes[0].set_ylabel("T [degC]"); axes[0].legend()
51         axes[0].set_title("TC_00: senal cruda con outliers")
52         axes[1].plot(df_10s.index, df_10s.mean(axis=1), 'b-'
, lw=1.0, label="media 10 canales")
53         axes[1].fill_between(df_10s.index,
df_10s.mean(axis=1) - df_10s.std(axis=1),
54         df_10s.mean(axis=1) + df_10s.std(axis=1),
55         alpha=0.25, label="+/- 1 sigma entre canales")
56         axes[1].set_ylabel("T [degC]"); axes[1].set_xlabel("
tiempo")
57
58         axes[1].legend(); axes[1].set_title("Promedio inter-
canales limpio (resampleo 10 s)")
59
60         plt.tight_layout()
61         plt.savefig("Figuras/fig_s2_pandas_pipeline.pdf")
62         plt.show()
63

```

Listing 15: Pipeline mínimo de limpieza y análisis con pandas.



2.12.2. Lectura metodológica

Tres operaciones no triviales aparecen en este ejemplo: detección de outliers con filtro de Hampel ($\text{score} > 3.5 \text{ MAD}$), imputación de NaN con interpolación temporal, y resampleo con `resample`. Estas operaciones son irreversibles: una vez aplicadas, el dato original ya no es recuperable salvo que se conserve por separado. La regla doctoral es: *nunca sobrescribir los datos crudos*; conservarlos en `data/raw/` como solo lectura y producir derivados en `data/processed/`.

2.13. Ejemplo 4: Diferenciación automática con `jax/autograd`

2.13.1. Planteamiento

La diferenciación automática (AD) calcula gradientes exactos hasta precisión de máquina, sin pasar por diferencias finitas (ruidosas) ni derivación simbólica (frágil ante simplificaciones). Es el motor detrás de PyTorch, TensorFlow, JAX y PINNs (que verán en la Semana 14).

Considere el ajuste no lineal de un modelo $f(x; \theta) = \theta_0 e^{-\theta_1 x} \cos(\theta_2 x + \theta_3)$ a datos ruidosos $\{(x_i, y_i)\}$. El gradiente de la pérdida cuadrática

$$\mathcal{L}(\theta) = \frac{1}{2} \sum_i (f(x_i; \theta) - y_i)^2 \quad (90)$$

respecto a $\theta \in \mathbb{R}^4$ se obtiene automáticamente con `jax.grad`, sin escribir derivadas a mano.

```

1      """
2      Ejemplo 4: ajuste no lineal con JAX y diferenciacion
3      automatica.
4      Modelo: f(x;theta) = theta0 * exp(-theta1 * x) * cos
5      (theta2*x + theta3)
6      """
7      import jax
8      import jax.numpy as jnp
9      import numpy as np
10     import matplotlib.pyplot as plt
11     from scipy.optimize import minimize
12
13     rng = np.random.default_rng(7)
14     theta_true = jnp.array([2.0, 0.3, 4.0, 0.5])
15
16     x = jnp.linspace(0, 6, 80)
    
```

```

16         def f(theta, x):
17             return theta[0] * jnp.exp(-theta[1]*x) * jnp.cos(
theta[2]*x + theta[3])
18
19         y_clean = f(theta_true, x)
20         y = y_clean + 0.1 * rng.normal(size=len(x))
21
22         # Funcion de perdida y su gradiente automatico
23         def loss(theta, x, y):
24             return 0.5 * jnp.sum((f(theta, x) - y)**2)
25
26         grad_loss = jax.grad(loss)
27
28         # Wrappers para scipy.optimize
29         def loss_np(theta):
30             return float(loss(jnp.array(theta), x, jnp.array(y))
)
31
32         def grad_np(theta):
33             return np.array(grad_loss(jnp.array(theta), x, jnp.
array(y)))
34
35         theta0 = np.array([1.0, 0.1, 3.0, 0.0])
36         res = minimize(loss_np, theta0, jac=grad_np, method=
"L-BFGS-B",
37             options={"gtol": 1e-9, "disp": True})
38
39         print("theta verdadero: ", np.asarray(theta_true))
40         print("theta estimado: ", res.x)
41         print("error parametrico:", np.linalg.norm(res.x -
np.asarray(theta_true)))
42
43         plt.figure(figsize=(8, 4))
44         plt.plot(x, y, 'k.', ms=4, label='datos ruidosos')
45         plt.plot(x, np.asarray(f(theta_true, x)), 'b-', lw
=1.0, label='modelo real')
46         plt.plot(x, np.asarray(f(jnp.array(res.x), x)), 'r--
', lw=1.2, label='ajuste L-BFGS-B')
47         plt.xlabel('x'); plt.ylabel('f(x)')
48         plt.title('Ajuste no lineal con gradientes via AD (
JAX)')
49
50         plt.grid(True, alpha=0.3); plt.legend()
51         plt.tight_layout()

```

```

50 plt.savefig("Figuras/fig_s2_jax_fit.pdf"); plt.show
51 ()

```

Listing 16: Diferenciación automática con JAX para ajuste no lineal.

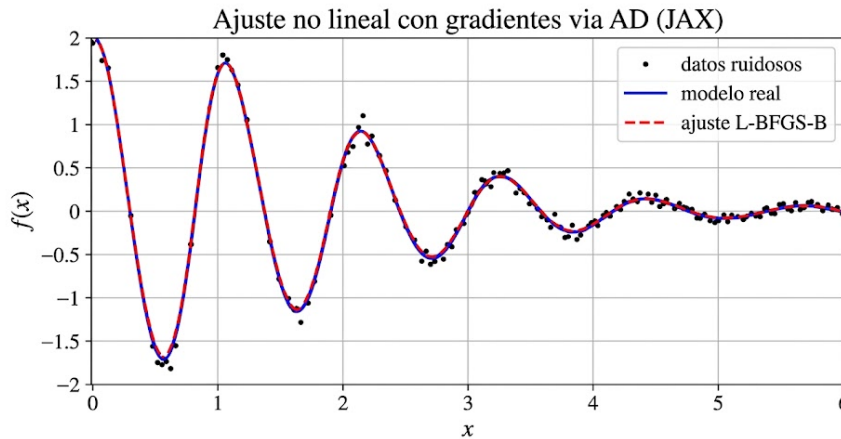


Figura 19: Ajuste no lineal por mínimos cuadrados con L-BFGS-B usando gradientes obtenidos por diferenciación automática (JAX). La curva ajustada (rojo) recupera la curva real (azul) hasta dentro del ruido. Sin AD, el usuario debería derivar a mano cuatro expresiones parciales y arriesgarse a errores de signo.

2.13.2. Lectura metodológica

La diferenciación automática es *distinta* de la derivación simbólica: AD computa el gradiente del programa exacto, sin construir ni simplificar la expresión algebraica. Es lineal en el costo (forward mode) o casi-constante (reverse mode/backprop) respecto a la dimensión de parámetros. En problemas con $\mathcal{O}(10^6)$ parámetros (redes neuronales) sería inviable hacerlo simbólicamente. Para PINNs, AD permite calcular $\partial u / \partial t$ y $\partial^2 u / \partial x^2$ de la red neuronal misma, lo cual conecta directamente con la Semana 14.

2.14. Ejemplo 5: Pruebas unitarias para código numérico

2.14.1. Planteamiento

Una función numérica sin pruebas es código no validado. La biblioteca estándar *de facto* en Python es `pytest`. Las pruebas mínimas para un módulo numérico son:

- Comparación contra solución analítica (cuando exista).
- Casos límite (entrada nula, parámetro extremo).
- Conservación (masa, energía, dimensiones).

- Orden de convergencia (la solución mejora como h^p).

```

1      # src/diffusion1d.py
2      import numpy as np
3
4      def step_diffusion(T, r):
5          """Un paso explícito de difusión 1D con fronteras
6          Dirichlet fijas.
7
8          Parameters
9          -----
10         T : ndarray, shape (N,)
11            Campo de temperatura actual [degC].
12         r : float
13            Numero de Fourier alpha*dt/dx^2, debe cumplir r <=
14            0.5.
15
16         Returns
17         -----
18         Tnew : ndarray, shape (N,)
19            Campo en el paso siguiente con fronteras preservadas
20            .
21
22         """
23         if not (0 < r <= 0.5):
24             raise ValueError(f"r={r} fuera del intervalo de
estabilidad (0, 0.5]")
25         Tnew = T.copy()
26         Tnew[1:-1] = T[1:-1] + r*(T[2:] - 2*T[1:-1] + T
[: -2])
27         return Tnew

```

Listing 17: Módulo y prueba unitaria con pytest para integrador de calor 1D.

```

1      # tests/test_diffusion1d.py
2      import numpy as np
3      import pytest
4      from src.diffusion1d import step_diffusion
5
6      def test_estabilidad_rechaza_r_grande():
7          with pytest.raises(ValueError):
8              step_diffusion(np.zeros(10), r=0.6)
9
10     def test_campo_constante_es_invariante():

```

```

11         T = np.full(50, 100.0)
12         T_new = step_diffusion(T, r=0.4)
13         assert np.allclose(T_new, T)
14
15     def test_fronteras_preservadas():
16         T = np.linspace(0, 1, 20)
17         T_new = step_diffusion(T, r=0.4)
18         assert T_new[0] == T[0]
19         assert T_new[-1] == T[-1]
20
21     def test_orden_convergencia_temporal():
22         """En estado estable lineal, residuo numerico  $O(h^2)$ 
23         . """
24         N = 101
25         x = np.linspace(0, 1, N)
26         T = x.copy() # solucion exacta de
27         Laplace 1D
28         T_new = step_diffusion(T, r=0.4)
29         err = np.max(np.abs(T_new - T))
30         assert err < 1e-12 # campo lineal:
31         residuo a precision maquina

```

Listing 18: Suite de pruebas pytest asociada.

Ejecución: `pytest tests/ -v`. Las cuatro pruebas anteriores tardan milisegundos y atrapan al menos cuatro clases de errores que han costado, históricamente, meses de retrabajo en tesis. La inversión es marginal; el ahorro, masivo.

2.15. Ejercicios guiados para uso de IA generativa

2.15.1. Principio operativo (repetido del Tema 1)

La IA generativa redacta código; no valida ciencia. El estudiante debe pedir, ejecutar, revisar, depurar e interpretar. Los prompts siguientes son ejercicios formales para la Semana 2.

2.15.2. Ejercicio IA-1: vectorización

Genera un script en Python que compare el tiempo de ejecución de calcular $y_i = \tanh(\alpha x_i) \sin(\beta x_i)$ para $i = 1, \dots, N = 10^7$ usando: (i) un bucle for explícito,

(ii) una expresión vectorizada NumPy, (iii) `numba.njit`. Usa $\alpha = 2.5$, $\beta = 7$, $x \in [-3, 3]$. Mide tiempo con `time.perf_counter`, imprime *speedup* relativo y verifica que las tres soluciones coinciden con `np.allclose`. Incluye comentarios técnicos en español.

2.15.3. Ejercicio IA-2: SVD aplicada

Genera un script en Python que cargue una imagen en escala de grises (puede ser `scipy.datasets.face(gray=True)`), calcule su SVD completa y muestre reconstrucciones truncadas con $k \in \{5, 20, 50, 100\}$. Para cada k , calcula el error relativo de Frobenius $\|A - A_k\|_F / \|A\|_F$ y la fracción de energía retenida. Grafica los cuatro resultados en una rejilla 2×2 con `viridis` desactivado (usa `gray`). Imprime una tabla con k , ratio de compresión y error.

2.15.4. Ejercicio IA-3: SciPy solve_ivp comparativo

Genera un script que integre el sistema de Lorenz $\dot{x} = \sigma(y - x)$, $\dot{y} = x(\rho - z) - y$, $\dot{z} = xy - \beta z$ con $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, $t \in [0, 40]$, $\mathbf{y}_0 = (1, 1, 1)$. Compara RK45 y DOP853 con tolerancias $\text{rtol} = \text{atol} = 10^{-6}$ y 10^{-10} . Reporta `nfev` y diferencia $\|y_{\text{RK45}} - y_{\text{DOP853}}\|_2$ al tiempo final. Grafica el atractor en 3D usando `mpl_toolkits.mplot3d`.

2.15.5. Ejercicio IA-4: pandas + visualización

Genera un script en Python que: (i) descargue el dataset de mediciones meteorológicas `seaborn.load_dataset("flights")` (o similar), (ii) cree un *heatmap* mensual usando `pivot`, (iii) calcule la media móvil de 12 meses, (iv) ajuste una regresión lineal con `statsmodels` y (v) genere una figura con dos paneles: *heatmap* y serie temporal con la regresión superpuesta. Usa configuración `rcParams` de publicación.

2.15.6. Ejercicio IA-5: script ejecutable + Makefile

Genera un proyecto Python con la siguiente estructura: `src/oscilador.py` con función `simular(m, k, c, x0, v0, T, dt)`; `scripts/run.py` con `argparse`; `Makefile` que ejecute tres experimentos con $\zeta \in \{0.05, 0.2, 0.8\}$ y produzca tres figuras y un `config.json` por experimento. Incluye un `README.md` con instrucciones, un `requirements.txt` con versiones fijas y un `.gitignore`.

2.15.7. Ejercicio IA-6: pruebas unitarias

Genera una suite `pytest` para una función `trapezoid(f, a, b, n)` que integra numéricamente. Las pruebas deben verificar: (i) integral de constante igual a longitud por valor; (ii) integral de x en $[0, 1]$ igual a $1/2$ con $n \geq 10$; (iii) orden de convergencia $\mathcal{O}(h^2)$ refinando $n \in \{10, 20, 40, 80\}$ y verificando que el error decae al menos como $h^{1.8}$; (iv) excepción si $a \geq b$. Comenta en español por qué cada prueba captura una clase distinta de error.

2.16. Actividad de refuerzo en clase: 30–40 minutos

2.16.1. Nombre de la actividad

Del cuaderno al script: refactor reproducible.

2.16.2. Duración

35 minutos: diagnóstico del notebook (5 min), refactor a script + `argparse` (15 min), prueba unitaria mínima (8 min), discusión (7 min).

2.16.3. Organización

Equipos de 2–3 personas. Cada equipo recibe un *notebook* “intencionalmente malo” (variables globales, sin semilla, gráficas sin unidades, ejecución de celdas en orden no lineal) que reproduce una versión simplificada de uno de los ejemplos de la Semana 1. La tarea es convertirlo en un script `run_experiment.py` con `argparse`, semilla, `config.json`, una figura “publication-ready” y al menos una prueba `pytest`.

2.16.4. Producto entregable

Una carpeta con: `run_experiment.py`, `config.json`, una figura PDF, `test_experiment.py` y un `README.md` de cinco líneas explicando cómo ejecutarlo.

2.16.5. Rúbrica rápida

Tabla 11: Rúbrica de evaluación rápida para la actividad de refactor.

Criterio	Excelente	Suficiente	Insuficiente
Estructura del proyecto	Carpetas <code>src/</code> , <code>scripts/</code> , <code>tests/</code> separadas.	Todo en raíz pero archivos identificables.	Un solo archivo o estructura caótica.
Parametrización	<code>argparse</code> completo y validación de rangos.	<code>argparse</code> básico.	Parámetros <i>hardcoded</i> .
Reproducibilidad	Semilla + <code>config.json</code> + versiones de librerías.	Semilla sin registro de versiones.	Sin semilla ni registro.
Figura	Unidades, viridis, PDF vectorial.	Unidades pero PNG no vectorial.	Sin unidades o jet.
Prueba unitaria	Al menos una prueba significativa que falla con bug introducido a propósito.	Prueba trivial (<code>assert True</code>).	Sin prueba.

2.17. Ejercicios para tarea

2.17.1. Ejercicio 1: refactor de la Semana 1 (Ejemplo 7)

Tome el código del Ejemplo 7 de la Semana 1 (conducción 2D en placa) y refáctorelo a un proyecto con la estructura sugerida en este tema: `src/`, `scripts/`, `tests/`, `config/`. Añada `argparse` para L_x , L_y , α , T_h , T_c , T_0 , tol_{ss} . Genere tres experimentos con `Makefile` y reporte un `config.json` por experimento.

2.17.2. Ejercicio 2: cuantificación de costo computacional

Implemente la suma $\sum_{i=1}^N \sin(2\pi i/N)$ de cuatro maneras: bucle `for`, comprensión de lista, `numpy.sum`, y suma con compensación de Kahan. Para $N \in \{10^3, 10^5, 10^7, 10^9\}$, mida tiempo y error absoluto contra el valor exacto (que es 0 por simetría). Discuta cuándo importa la precisión y cuándo el tiempo.

2.17.3. Ejercicio 3: estabilidad numérica

Calcule el determinante de la matriz de Hilbert H_n , $H_{ij} = 1/(i+j-1)$, para $n = 5, 10, 15, 20$, usando: (i) `numpy.linalg.det` sobre `float64`; (ii) `numpy.linalg.det` sobre `float32`; (iii) `sympy.Matrix(...).det()` en aritmética racional. Compare. Explique por qué `float32` falla mucho antes que `float64` y por qué `SymPy` no falla nunca para este problema (pero es lentísimo).

2.17.4. Ejercicio 4: visualización rigurosa

Tome una figura publicada en alguno de los artículos de la Semana 1 (apéndice) y rediseñela siguiendo los criterios de la [Figura 14](#): unidades, mapa de color perceptual, tipografía consistente, aspect ratio físico. Entregue antes y después.

2.17.5. Ejercicio 5: pandas sobre datos reales

Descargue un dataset público (por ejemplo, registros climáticos de NOAA, datos COVID de Our World in Data, o cualquier CSV de su tema doctoral). Construya un pipeline pandas de al menos cuatro pasos: lectura, limpieza de NaN, detección de outliers, re-samplado/agregación. Documente cada paso con su efecto cuantitativo (cuántos NaN se eliminaron, cuántos outliers, etc.). Conserve el archivo original como solo lectura.

2.17.6. Ejercicio 6: diferenciación automática

Use JAX o PyTorch para calcular el gradiente de $\mathcal{L}(\theta) = \|f(x; \theta) - y\|^2$ con f una red neuronal de tres capas con activaciones tanh, ajustando a 50 puntos de la función $y = \sin(2\pi x)$ con ruido. Reporte la evolución de la pérdida durante 1000 iteraciones de Adam. Discuta por qué AD escala bien con la dimensión de θ mientras que diferencias finitas no.

2.17.7. Ejercicio 7: pruebas unitarias completas

Para el código del Ejemplo 8 de la Semana 1 (FEM CST), escriba al menos cinco pruebas pytest: (i) matriz de rigidez simétrica; (ii) campo de desplazamiento nulo si la carga es nula; (iii) magnitud del desplazamiento aumenta linealmente con la carga (régimen elástico); (iv) verificación contra cantilever analítico ($u = PL^3/(3EI)$ aproximado); (v) error de redondeo en la imposición de Dirichlet inferior a 10^{-10} del rango de la solución.

2.17.8. Ejercicio 8: tu repositorio doctoral

Cree un repositorio público en GitHub o GitLab para su proyecto de tesis (puede ser un repositorio privado durante el desarrollo y volverse público al final). Inicialícelo con: README.md, LICENSE, environment.yml, .gitignore, estructura de carpetas del tema, y al menos un *commit* con un *tag* v0.1-setup. Comparta el enlace para revisión. Si el repositorio no existe, no existe el proyecto.

2.18. Cierre conceptual

El contenido de esta semana se resume en una afirmación simple: la herramienta computacional no es accesoria, es parte de la metodología científica. Las siete reglas que se exigirán en los entregables de aquí en adelante son:

1. Todo proyecto vive bajo control de versiones desde el primer día.
2. Todo experimento numérico tiene una semilla aleatoria explícita.
3. Todo parámetro entra por línea de comandos o por archivo de configuración, nunca *hardcoded*.
4. Toda figura tiene unidades, mapa de color perceptual y exportación vectorial.
5. Toda función numérica crítica tiene al menos una prueba unitaria.
6. Todo entorno está congelado con versiones fijas (`environment.yml` o `requirements.txt`).
7. Toda conclusión es trazable a un *commit* específico y a un `config.json` específico.

La Semana 3 entrará formalmente en métodos numéricos fundamentales: diferencias finitas, integración, derivación, estabilidad, convergencia y condicionamiento. La infraestructura construida en esta semana es la que permitirá que esos métodos se implementen, se prueben y se comparen sin reinventar la base operativa en cada problema.

Apéndice Semana 2. Referencias verificables

- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). *SciPy 1.0: fundamental algorithms for scientific computing in Python*. Nature Methods, 17, 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. Computing in Science & Engineering, 9(3), 90–95. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- McKinney, W. (2010). *Data structures for statistical computing in Python*. Proceedings of the 9th Python in Science Conference (SciPy 2010), 56–61. DOI: [10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a).
- Bradbury, J., Frostig, R., Hawkins, P., et al. (2018). *JAX: composable transformations of Python+NumPy programs*. github.com/google/jax.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. jmlr.org/papers/v12/pedregosa11a.html.
- Wilson, G., Aruliah, D. A., Brown, C. T., et al. (2014). *Best practices for scientific computing*. PLOS Biology, 12(1), e1001745. DOI: [10.1371/journal.pbio.1001745](https://doi.org/10.1371/journal.pbio.1001745).

- Wilson, G., Bryan, J., Cranston, K., et al. (2017). *Good enough practices in scientific computing*. PLOS Computational Biology, 13(6), e1005510. DOI: [10.1371/journal.pcbi.1005510](https://doi.org/10.1371/journal.pcbi.1005510).
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). *Ten simple rules for reproducible computational research*. PLOS Computational Biology, 9(10), e1003285. DOI: [10.1371/journal.pcbi.1003285](https://doi.org/10.1371/journal.pcbi.1003285).
- Crameri, F., Shephard, G. E., & Heron, P. J. (2020). *The misuse of colour in science communication*. Nature Communications, 11, 5444. DOI: [10.1038/s41467-020-19160-7](https://doi.org/10.1038/s41467-020-19160-7).
- Mölder, F., Jablonski, K. P., Letcher, B., et al. (2021). *Sustainable data analysis with Snakemake*. F1000Research, 10:33. DOI: [10.12688/f1000research.29032.2](https://doi.org/10.12688/f1000research.29032.2).
- Perez, F., & Granger, B. E. (2007). *IPython: A system for interactive scientific computing*. Computing in Science & Engineering, 9(3), 21–29. DOI: [10.1109/MC-SE.2007.53](https://doi.org/10.1109/MC-SE.2007.53).
- Kluyver, T., Ragan-Kelley, B., Pérez, F., et al. (2016). *Jupyter Notebooks – a publishing format for reproducible computational workflows*. En Loizides & Schmidt (eds.), *Positioning and Power in Academic Publishing*, 87–90. DOI: [10.3233/978-1-61499-649-1-87](https://doi.org/10.3233/978-1-61499-649-1-87).
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., et al. (2016). *The FAIR Guiding Principles for scientific data management and stewardship*. Scientific Data, 3, 160018. DOI: [10.1038/sdata.2016.18](https://doi.org/10.1038/sdata.2016.18).